

Volume 4

GitHub Pages and Ontology Site Maintenance

Vocabulary Deployment, HTML Generation, Redirect Infrastructure, and CSS

This volume follows:

Volume 1 — From Source Text to Structured Dataset

Volume 2 — From Spreadsheet to Linked Data

Volume 3 — The Mapping Workbook

Iroko Historical Society | irokosociety.org | March 2026

Introduction

This volume documents the infrastructure that makes the Iroko Framework publicly accessible: the GitHub repository, the GitHub Pages deployment, the Python scripts that generate HTML browse pages from TTL vocabulary files, the `iroko.html` redirect file that makes every term URI dereferenceable, the CSS and assets that style the site, and the whitepaper deployment. It also documents what not to do — the set of actions that look like maintenance but cause cascading problems if performed incorrectly.

The ontology site lives at two addresses that point to the same GitHub Pages deployment:

- <https://ontology.irokosociety.org> — the public-facing canonical address, routed via CNAME
- <https://iroko-framework.github.io/iroko-framework/> — the underlying GitHub Pages URL

The main institution website at irokosociety.org is hosted on Squarespace and is not covered by this volume. The two sites are independent. Changes to the GitHub Pages deployment do not affect Squarespace and vice versa.

This volume is written for anyone who needs to deploy a vocabulary update, fix a site issue, or understand how the pieces fit together. No prior GitHub or web deployment experience is assumed.

Part 1: Repository Structure

Where the repository lives

The Iroko Framework vocabulary repository is hosted on GitHub at:

```
https://github.com/iroko-framework/iroko-framework
```

GitHub Pages serves the repository contents at the GitHub Pages URL above. The CNAME file in the repository root tells GitHub Pages to also serve the site at the canonical domain ontology.iroko.org. Both addresses resolve to the same content.

Top-level directory structure

Path	Purpose
/ (repo root)	iroko.html redirect, CNAME, index.html (the site landing page)
/vocab/	All 16 TTL vocabulary files and their generated HTML browse pages. Also contains iroko-termlist.html (the full term index).
/assets/	iroko-style.css (shared stylesheet), IHS-Logo.jpg (the logo used on all browse pages and the whitepaper sidebar)
/scripts/	generate_vocab_html.py and generate_vocab_index.py — the two Python generation scripts
/whitepaper/	index.html — the Iroko Framework white paper deployed as a standalone page

The vocab/ directory in detail

The vocab/ directory is the core of the deployment. It contains two types of files for each module: the source TTL file and the generated HTML browse page. The HTML is always generated from the TTL — never edited manually.

File	Type	Notes
iroko-core.ttl	Source	Core vocabulary — access levels, assertions, provenance
iroko-core.html	Generated	Do not edit manually. Regenerate using generate_vocab_html.py
iroko-ewe.ttl	Source	Ewé module — sacred plant concepts (medicinal, ritual, harvest)
iroko-ewe.html	Generated	Do not edit manually

File	Type	Notes
iroko-agency.ttl	Source	Agency module — sacred agents, events, authorization
iroko-agency.html	Generated	Do not edit manually
iroko-authority.ttl	Source	Authority module — ritual governance
iroko-authority.html	Generated	Do not edit manually
iroko-epistemic.ttl	Source	Epistemic module — knowledge gating
iroko-epistemic.html	Generated	Do not edit manually
iroko-narrative.ttl	Source	Narrative module — transmission chains, variant relations
iroko-narrative.html	Generated	Do not edit manually
iroko-manifestation.ttl	Source	Manifestation module — possession, dream, divination modes
iroko-manifestation.html	Generated	Do not edit manually
iroko-nkisi.ttl	Source	Nkisi module — spiritual entities
iroko-nkisi.html	Generated	Do not edit manually
iroko-travay.ttl	Source	Travay module — ritual processes, ceremonies
iroko-travay.html	Generated	Do not edit manually
iroko-ile.ttl	Source	Ilé module — houses, lineage, religious office
iroko-ile.html	Generated	Do not edit manually
iroko-marca.ttl	Source	Marca module — divination systems, Odu
iroko-marca.html	Generated	Do not edit manually
iroko-ekpe.ttl	Source	Ékpè module — initiatory societies
iroko-ekpe.html	Generated	Do not edit manually
iroko-veve.ttl	Source	Vèvè module — graphic sign systems
iroko-veve.html	Generated	Do not edit manually
iroko-ngoma.ttl	Source	Ngoma module — sacred music, Añá
iroko-ngoma.html	Generated	Do not edit manually
iroko-sankofa.ttl	Source	Sankofa module — reclamation movements
iroko-sankofa.html	Generated	Do not edit manually
iroko-qal.ttl	Source	Qal module — sacred lexicons, liturgical language
iroko-qal.html	Generated	Do not edit manually
iroko-termlist.html	Generated	Full term index — all 1,044 terms across all 16 modules. Do not edit manually.
index.html	Maintained	The vocabulary landing page listing all modules.

WARNING: *The HTML files in vocab/ are generated outputs. Editing them manually is futile — the next time generate_vocab_html.py runs for that module, the manual edit will be overwritten. All content changes must be made in the TTL source file. Presentational changes must be made in iroko-style.css.*

Files that are always manually maintained

These files are never generated by a script and require human editing:

- `iroko.html` — the term redirect file at the repository root. Must be updated manually whenever new terms are added to any module.
- `CNAME` — one line: `ontology.irokosociety.org`. Do not edit this.
- `assets/iroko-style.css` — the shared stylesheet for all browse pages. Edit for presentational changes only.
- `assets/IHS-Logo.jpg` — the logo image. Replace only if the logo changes.
- `whitepaper/index.html` — the whitepaper. Updated when a new version of the white paper is released.
- `vocab/index.html` — the vocabulary landing page. Update when new modules are added or module descriptions change.

Part 2: The Vocabulary Update Workflow

Overview

A vocabulary update means a change to one or more TTL files: adding new concepts, modifying existing definitions, adding new properties or classes, or correcting errors. The update workflow has four stages: edit the TTL, run the generation scripts, update `iroko.html` if new terms were added, and push to GitHub. GitHub Pages deploys automatically on push — there is no separate deployment step.

Step 2.1 — Edit the TTL file

Open the relevant TTL file in a text editor. TTL (Turtle) is a human-readable RDF serialization format. The Iroko Framework uses a consistent authoring style — follow existing conventions when adding new content.

Basic Turtle syntax reminders:

```
# A new concept in a ConceptScheme
iroko:medicinal-wound-healing
  a skos:Concept ;
  skos:prefLabel "Wound Healing"@en ;
```

```

    skos:definition "Use for plants documented as treating cuts, wounds, sores, or
skin injuries."@en ;
    skos:inScheme iroko:MedicinalUseScheme ;
    iroko:minimumAccessLevel iroko:access-public-unrestricted .

# A new property
iroko:vodouName
    a owl:AnnotationProperty ;
    rdfs:label "Vodou Name"@en ;
    rdfs:comment "Haitian Vodou liturgical or ceremonial name for this entity."@en
;
    rdfs:domain iroko:EwePlantRecord .

```

WARNING: Always validate the TTL file before running the generation scripts. A syntax error in the TTL will cause rdflib to fail silently or raise an exception, and the corresponding HTML file will not be regenerated. Use a TTL validator (*rapper*, *riot*, or the online *EasyRDF converter*) to check syntax before proceeding.

To validate a TTL file from the command line:

```

# Using rapper (part of the Raptor RDF library)
rapper -i turtle -o ntriples iroko-ewe.ttl > /dev/null

# Using riot (part of Apache Jena)
riot --validate iroko-ewe.ttl

# Using Python rdflib directly
python3 -c "from rdflib import Graph; g = Graph(); g.parse('iroko-ewe.ttl',
format='turtle'); print(f'{len(g)} triples parsed successfully')"

```

Step 2.2 — Run the HTML generation scripts

The generation scripts read TTL files and produce HTML browse pages. They must be run from the `scripts/` directory, or from any directory where the relative path to `vocab/` resolves correctly. The scripts expect to find the TTL files in a sibling `vocab/` directory.

`generate_vocab_html.py` — regenerate browse pages

This script generates one HTML browse page per module. Run it after any TTL change.

```

# Regenerate a single module (recommended — faster, safer)
python3 scripts/generate_vocab_html.py iroko-ewe

# Regenerate multiple modules
python3 scripts/generate_vocab_html.py iroko-ewe iroko-core

# Regenerate all 16 modules (use only when making cross-cutting changes)
python3 scripts/generate_vocab_html.py

```

The script prints a summary for each module processed:

```
Processing iroko-ewe.ttl ...
v iroko-ewe.html (0 cls, 12 prop, 8 sch, 62 concepts)
```

If a module is not in `MODULE_CONFIG`, the script skips it and prints `SKIPPED`. This is expected behavior for legacy or alignment files. The intentionally excluded files are documented in the script header:

```
# TTLS intentionally excluded (no MODULE_CONFIG entry):
# iroko-align-prov - reference-only PROV-O alignment; 0 user-facing terms
# iroko-nkisi-patch - broken syntax, superseded
# ewe-plants-v0_2_1 - legacy instance data file, not a vocabulary
# iroko-vocab-v0_2_1 - superseded version
```

generate_vocab_index.py — regenerate the full term index

This script reads all 16 active TTL modules and generates `iroko-termlist.html`, the searchable full-term index. Run it after any TTL change that adds, removes, or renames terms, since the term index must stay current.

```
# Standard usage - writes to vocab/iroko-termlist.html
python3 scripts/generate_vocab_index.py

# Explicit output path
python3 scripts/generate_vocab_index.py --output vocab/iroko-termlist.html

# Read TTLS from a different directory
python3 scripts/generate_vocab_index.py --ttl-dir /path/to/vocab
```

NOTE: `generate_vocab_index.py` requires `rdflib`. Install it with: `pip3 install rdflib --break-system-packages`. The script also requires that all 16 active TTL files parse without errors. If any TTL has a syntax error, the index generation will fail or produce an incomplete index.

Step 2.3 — Decide whether to regenerate all modules or just one

Regenerating a single module is always preferable to regenerating all modules. Running all modules takes longer, increases the commit diff (making it harder to review changes), and risks overwriting a correctly generated file with a newly generated one if a TTL has an undetected issue. The rule is:

- **Edit one TTL** → **regenerate one module** using the module stem argument.
- **Edit multiple TTLS** → **regenerate only those modules** by passing multiple stems.
- **Run all modules** only when a cross-cutting change affects `MODULE_CONFIG` itself, the `iroko-style.css` is being tested, or a fresh full deployment is needed after a repository clone.

Part 3: The iroko.html Redirect File

What it does

When a browser navigates to a Iroko term URI such as:

```
https://ontology.irokosociety.org/iroko#medicinal-respiratory-support
```

GitHub Pages receives a request for the path /iroko. It finds the file iroko.html at the repository root and serves it. The JavaScript inside iroko.html reads the fragment identifier (#medicinal-respiratory-support) from the URL, looks it up in a term-to-module mapping table, and redirects the browser to the correct location:

```
https://ontology.irokosociety.org/vocab/iroko-ewe.html#concept-medicinal-respiratory-support
```

Without this file, every Iroko term URI would return a 404. The file is therefore a critical piece of infrastructure. Every term added to any module must be added to the mapping table in iroko.html before the term's URI is fully dereferenceable.

The mapping table structure

The mapping table is a JavaScript object with 1,044 entries as of the current deployment. Each entry maps a term identifier to the HTML file that contains that term's documentation:

```
const termMap = {  
  "concept-medicinal-respiratory-support": "iroko-ewe.html",  
  "concept-access-community-only":       "iroko-core.html",  
  "cls-Assertion":                       "iroko-core.html",  
  "prop-accessLevel":                    "iroko-core.html",  
  // ... 1,040 more entries  
};
```

The key format depends on term type:

- `concept-[slug]` — for SKOS concepts (e.g., `concept-medicinal-digestive-support`)
- `cls-[ClassName]` — for OWL classes (e.g., `cls-EwePlantRecord`)
- `prop-[propertyName]` — for OWL properties (e.g., `prop-accessLevel`)

The redirect logic tries four fallbacks in sequence, so a bare slug without a type prefix will also resolve:

```
// 1. Try direct match: termMap[fragment]  
// 2. Try concept- prefix: termMap['concept-' + fragment]
```

```
// 3. Try cls- prefix: termMap['cls-' + fragment]
// 4. Try prop- prefix: termMap['prop-' + fragment]
// 5. Show not-found message with link to iroko-termlist.html
```

This means that the URI <https://ontology.irokosociety.org/iroko#medicinal-respiratory-support> resolves correctly whether the fragment is the full prefixed form `concept-medicinal-respiratory-support` or the bare slug `medicinal-respiratory-support`. Both are handled.

Updating iroko.html when new terms are added

Every time a new concept, class, or property is added to any TTL module, a corresponding entry must be added to the `termMap` in `iroko.html`. The process:

1. Identify the term type (concept, class, or property) and the HTML file it appears in after regeneration.
2. Open `iroko.html` in a text editor. Find the `const termMap = {` line.
3. Add the new entry in alphabetical order within the map. The entries are alphabetically sorted to make manual review easier.
4. Verify the HTML filename is correct by checking that the corresponding browse page was generated and contains the anchor.

```
// Adding a new Ewé concept: iroko:medicinal-wound-healing
// In iroko-ewe.html it will appear as id="concept-medicinal-wound-healing"

// Find the correct alphabetical position and insert:
"concept-medicinal-wound-healing": "iroko-ewe.html",

// Adding a new property: iroko:vodouName
// In iroko-ewe.html it will appear as id="prop-vodouName"
"prop-vodouName": "iroko-ewe.html",
```

NOTE: The term identifier in the `iroko.html` map must exactly match the `id` attribute generated in the HTML browse page. The HTML generation script uses the local name of the URI as the `id` — so `iroko:medicinal-wound-healing` becomes `concept-medicinal-wound-healing`, `iroko:EwePlantRecord` becomes `cls-EwePlantRecord`, and `iroko:vodouName` becomes `prop-vodouName`. Check the generated HTML to confirm the `id` before adding the map entry.

Verifying a term resolves correctly

After pushing a change that adds new terms to `iroko.html`, verify the redirect works:

5. Wait 1–2 minutes for GitHub Pages to deploy (it deploys automatically on push but is not instant).
6. Navigate to the term URI in a browser:
[https://ontology.irokosociety.org/iroko#\[your-new-slug\]](https://ontology.irokosociety.org/iroko#[your-new-slug])

7. Confirm the browser redirects to the correct module browse page and scrolls to the correct term.
8. If the redirect fails, check: (1) the iroko.html entry was saved and pushed, (2) the term id in the HTML matches the map key exactly, (3) GitHub Pages has finished deploying.

TIP: To see whether GitHub Pages has deployed your push, go to the repository on GitHub, click *Settings > Pages*, and look at the deployment status. A green checkmark means the latest push is live.

Part 4: GitHub Pages Configuration

How GitHub Pages is configured

GitHub Pages is configured in the repository settings to serve from the main branch, root directory. This means the repository root is the web root — any file in the repository is accessible at the corresponding URL path. The configuration should not need to change.

To verify the current configuration:

9. Go to the repository on GitHub.
10. Click Settings > Pages.
11. Confirm: Source is **"Deploy from a branch"**, Branch is **main**, Folder is **/ (root)**
12. The Custom domain field should show: ontology.irokosociety.org

The CNAME file

The CNAME file at the repository root contains exactly one line:

```
ontology.irokosociety.org
```

This file tells GitHub Pages to serve the site at the custom domain. Do not edit this file. Do not delete this file. If it is accidentally deleted, GitHub Pages will stop serving the site at the custom domain and will only be accessible at the github.io URL. Restore it by creating a new file with the single line above.

The DNS record on the domain registrar side must point ontology.irokosociety.org to GitHub Pages' servers. This is a CNAME record pointing to iroko-framework.github.io. This was configured once and should not need to change. If the site stops resolving:

13. Check the DNS record is still in place at the domain registrar.
14. Check the CNAME file still exists in the repository root.
15. Check GitHub Pages settings show Enforce HTTPS is checked.

Enforce HTTPS

The Enforce HTTPS option should always be checked in GitHub Pages settings. This ensures all traffic is redirected from `http://` to `https://` automatically. If it is unchecked, check it and allow a few minutes for GitHub to provision an HTTPS certificate.

The Squarespace relationship

The main institution website at `irokosociety.org` is hosted on Squarespace. The ontology subdomain (`ontology.irokosociety.org`) is served by GitHub Pages. These are completely independent deployments with separate DNS records. A change to the GitHub Pages deployment does not affect Squarespace and vice versa. The shared visual element between the two sites is the IHS logo, which must be kept in sync manually if it ever changes.

Part 5: The HTML Generation Scripts in Detail

`generate_vocab_html.py`

This script is the primary tool for deploying vocabulary changes. It reads one or more TTL files from the `vocab/` directory and generates the corresponding HTML browse pages. Understanding how it works makes it easier to diagnose problems when the output is not what is expected.

How the script finds files

The script resolves file paths relative to its own location. When run from the `scripts/` directory, it looks for TTL files in the sibling `vocab/` directory (`../vocab/` relative to the script). When run from the repository root, it looks for `vocab/` as a subdirectory of the current working directory. If neither path exists, the script exits with an error.

```
# Run from repository root (recommended)
cd /path/to/iroko-framework
python3 scripts/generate_vocab_html.py iroko-ewe

# Run from scripts/ directory (also works)
cd /path/to/iroko-framework/scripts
python3 generate_vocab_html.py iroko-ewe
```

What the script generates

For each TTL file processed, the script generates an HTML file with the same stem and writes it to the same directory as the TTL (`vocab/`). It extracts:

- **Classes** (owl:Class definitions) — displayed as cards with subclass hierarchy
- **Properties** (owl:ObjectProperty, owl:DatatypeProperty, owl:AnnotationProperty) — displayed as a table with type, domain, range, access badge, and description

- **Concept Schemes and Concepts** (skos:ConceptScheme and skos:Concept) — grouped by scheme, displayed with slug, prefLabel, definition, altLabels, and scope notes

The language preference for all extracted labels is English (@en). If no English label exists, the script falls back to any available language tag. If no label exists at all, the script falls back to the local name of the URI.

MODULE_CONFIG — what controls which files are processed

The MODULE_CONFIG dictionary at the top of the script controls which TTL stems are recognized and what metadata (title, subtitle, tag class, module prefix) is used in the generated HTML. If a TTL file exists in vocab/ but is not in MODULE_CONFIG, the script skips it with a SKIPPED message. This is intentional for legacy and excluded files.

When a new module is added to the Iroko Framework, it must be added to MODULE_CONFIG before the script can generate its browse page. The entry format is:

```
"iroko-newmodule": {
  "title": "New Module — Brief Description",
  "subtitle": "Longer subtitle explaining scope",
  "tag_cls": "tag-newmodule",      # must match a CSS class in iroko-style.css
  "tag_text": "Tag Label",
  "prefix": "nm:",                 # the namespace prefix for this module
},
```

NOTE: The tag_cls value must match a .tag-* CSS class defined in iroko-style.css. If the CSS class does not exist, the module tag will render without styling. Add the CSS class to iroko-style.css before adding the module to MODULE_CONFIG.

generate_vocab_index.py

This script reads all 16 active TTL modules (defined in its internal MODULES list) and generates a single-page searchable term index at vocab/iroko-termlist.html. The index includes every class, property, and concept across all modules with live search filtering, type facets (Class, Property, Concept), and module facets.

The script uses the same anchor naming convention as the browse page generator: cls- for classes, prop- for properties, concept- for concepts. Each entry in the index links directly to the corresponding anchor in the module browse page, so clicking a term in the index takes the user directly to the term's documentation.

When the MODULES list needs updating

The MODULES list in generate_vocab_index.py is separate from MODULE_CONFIG in generate_vocab_html.py. Both must be updated when a new module is added. The MODULES list format is:

```
# (display_name, tier, tag_label, namespace_uri, ttl_stem)
```

```
("NewModule", "Domain", "Tag", "https://ontology.iroko-society.org/iroko#", "iroko-newmodule"),
```

Dependencies

Both scripts require `rdflib`. No other external dependencies are required. The HTML output uses inline styles and links to `iroko-style.css` — there are no JavaScript dependencies for the browse pages themselves (the term index page uses inline JavaScript for the search filter).

```
pip3 install rdflib --break-system-packages

# Verify installation
python3 -c "import rdflib; print(rdflib.__version__)"
```

Part 6: CSS and Assets

`iroko-style.css`

The shared stylesheet at `assets/iroko-style.css` controls the visual presentation of all 17 HTML browse pages and the term index. It is not generated — it is maintained manually. Changes take effect across all browse pages on the next push, without regenerating any HTML.

When to edit `iroko-style.css`

- Adding a new module tag color (a new `.tag-*` class for a new module).
- Fixing a layout or rendering issue visible across all browse pages.
- Updating typography, spacing, or color variables.
- The overflow: hidden → overflow: clip fix documented in Vol 2 Part 6 is an example of a CSS-only fix that did not require HTML regeneration.

When NOT to edit `iroko-style.css`

- To change the content of a term definition or label. That change belongs in the TTL.
- To add a module to the browse pages. That requires `MODULE_CONFIG`, `generate_vocab_html.py`, and `iroko.html`.
- To fix a typo in a term name. That belongs in the TTL.

RULE: CSS changes are presentational and independent of the vocabulary pipeline. You can commit and push a CSS change without running either Python script. Do not run the generation scripts for a CSS-only change — it creates unnecessary commits and complicates the change history.

The overflow: clip fix — permanent record

This fix was applied during the namespace migration and is documented here as a permanent record. The `.scheme-block` CSS class previously used `overflow: hidden`, which creates a new scroll container in browsers and prevents anchor navigation from scrolling to elements inside the container. The correct value is `overflow: clip`, which preserves the visual clipping behavior without creating a scroll container. The current stylesheet has this fix applied. Do not revert it.

```
/* CORRECT — do not change */
.scheme-block {
  overflow: clip;
}

/* WRONG — do not revert to this */
.scheme-block {
  overflow: hidden;
}
```

Module tag CSS classes

Each module has a corresponding CSS class in the format `.tag-[name]` that controls the color of the module badge displayed on its browse page. The current classes are:

```
.tag-core           { /* Foundation layer — blue-navy */ }
.tag-agency         { /* Governance layer */ }
.tag-authority      { /* Governance layer */ }
.tag-epistemic      { /* Governance layer */ }
.tag-narrative      { /* Governance layer */ }
.tag-manifestation { /* Governance layer */ }
.tag-botanical      { /* Ewe module */ }
.tag-entities       { /* Nkisi module */ }
.tag-ritual         { /* Travay module */ }
.tag-lineage        { /* Ile module */ }
.tag-divination     { /* Marca module */ }
.tag-societies      { /* Ekpe module */ }
.tag-graphic        { /* Veve module */ }
.tag-music          { /* Ngoma module */ }
.tag-movements      { /* Sankofa module */ }
.tag-language       { /* Qal module */ }
.tag-upcoming       { /* Placeholder for modules in development */ }
```

IHS-Logo.jpg

The logo is referenced from the HTML browse pages as `../assets/IHS-Logo.jpg` (relative to the `vocab/` directory) and from the whitepaper as `assets/IHS-Logo.jpg` (relative to the repo root). If the logo file is ever replaced, both references remain valid as long as the replacement file uses the same filename. If the filename changes, every reference in every generated HTML file and in the whitepaper must be updated.

NOTE: The logo has a CSS variable `--logo-size` set to 256px in the browse page stylesheet inline block. This controls the display size of the logo in the vocab page header. Do not change the actual image dimensions to resize the logo — change the CSS variable instead.

Part 7: The Whitepaper Deployment

How the whitepaper is deployed

The Iroko Framework white paper is deployed as `whitepaper/index.html` in the repository. It is accessible at:

```
https://ontology.irokosociety.org/whitepaper
```

The whitepaper HTML file is self-contained — all CSS is inline, all JavaScript is inline, and the only external dependencies are Google Fonts (loaded via a link element) and the IHS logo image (loaded from `assets/IHS-Logo.jpg` relative to the repo root).

Updating the whitepaper

When a new version of the white paper is ready:

16. The white paper is authored in its final HTML form, following the styling conventions of the existing `whitepaper/index.html`.
17. Replace `whitepaper/index.html` with the new version.
18. Update the DOI citation metadata in the `<head>` element if a new Zenodo DOI has been issued for the new version.
19. Commit and push. GitHub Pages deploys automatically.

The whitepaper's sidebar TOC uses a JavaScript IntersectionObserver for active-section highlighting. This is fully self-contained and requires no maintenance. The sticky sidebar requires no maintenance. The print stylesheet is embedded in the file.

NOTE: The whitepaper uses the `docx_to_html.py` converter documented in the *Technical Series* for authoring. The converter produces HTML in the whitepaper style. After conversion, review the output for: correct logo path (`assets/IHS-Logo.jpg`), correct DOI in the metadata block, and correct sidebar TOC entries. These may need manual adjustment after conversion.

The Zenodo DOI

The current white paper is registered at Zenodo with DOI 10.5281/zenodo.18826673. This DOI is embedded in the whitepaper HTML as a citation_doi meta tag and is displayed in the sidebar footer. When a new major version of the white paper is published, a new Zenodo deposit should be made and the DOI updated in the HTML. Minor corrections do not require a new Zenodo deposit.

Part 8: Complete Deployment Workflows

Workflow A — Adding concepts to an existing module

Example: adding iroko:medicinal-wound-healing to iroko-ewe.ttl.

20. **Edit the TTL.** Open vocab/iroko-ewe.ttl. Add the new concept following the existing style.
21. **Validate the TTL.** Run: `python3 -c "from rdflib import Graph; g = Graph(); g.parse('vocab/iroko-ewe.ttl', format='turtle'); print(len(g), 'triples')"`
22. **Regenerate the module browse page.** Run: `python3 scripts/generate_vocab_html.py iroko-ewe`
23. **Regenerate the term index.** Run: `python3 scripts/generate_vocab_index.py`
24. **Update iroko.html.** Add: "concept-medicinal-wound-healing": "iroko-ewe.html", in alphabetical position.
25. **Update the mapping workbook.** Add the new value to the Medicinal Uses sheet following the process in Vol 3 Part 9.
26. **Update Vol 2 GREL.** Add the new label-to-slug mapping to the GREL Quick Reference in Vol 2 Appendix A.
27. **Commit and push.** Include all changed files: iroko-ewe.ttl, iroko-ewe.html, iroko-termindex.html, iroko.html.
28. **Verify.** Navigate to <https://ontology.irokosociety.org/iroko#medicinal-wound-healing> and confirm the redirect works.

Workflow B — CSS presentational change only

Example: adjusting spacing or colors in iroko-style.css.

29. Edit assets/iroko-style.css.
30. Do not run generate_vocab_html.py.
31. Commit and push.
32. Verify visually at ontology.irokosociety.org/vocab/iroko-ewe.html after deployment.

Total files changed: 1. No scripts run.

Workflow C — Adding a new module to the framework

This is the most complex deployment operation. Example: adding a new Domain module iroko-newmod.

33. **Define the TTL.** Create vocab/iroko-newmod.ttl with the new module's ontology declaration, classes, properties, and concept schemes.
34. **Validate the TTL.** Run the rdflib validation command.
35. **Add to MODULE_CONFIG.** Add the new module entry to scripts/generate_vocab_html.py following the format in Part 5.
36. **Add to MODULES list.** Add the new module to the MODULES list in scripts/generate_vocab_index.py.
37. **Add CSS tag class.** Add the .tag-newmod CSS class to assets/iroko-style.css.
38. **Generate the browse page.** Run: python3 scripts/generate_vocab_html.py iroko-newmod
39. **Regenerate the term index.** Run: python3 scripts/generate_vocab_index.py
40. **Update iroko.html.** Add entries for every term in the new module.
41. **Update vocab/index.html.** Add the new module to the vocabulary landing page.
42. **Update the mapping workbook.** Add any new controlled vocabulary concepts following Vol 3 Part 9.
43. **Commit and push.** All changed files.
44. **Verify.** Check the new module browse page, the term index, and at least one term URI redirect.

Workflow D — White paper update

45. Convert the new white paper version to HTML using docx_to_html.py.
 46. Review and adjust the output: logo path, DOI, sidebar TOC.
 47. Replace whitepaper/index.html with the new file.
 48. If this is a new major version, make a new Zenodo deposit and update the citation_doi meta tag.
 49. Commit and push.
 50. Verify at <https://ontology.irokosociety.org/whitepaper>
-

Part 9: Troubleshooting

A term URI returns 404

The term's module HTML file may not be deployed, or the term may not be in the iroko.html map. Steps:

51. Confirm the corresponding HTML browse page exists in vocab/ and has been pushed.
52. Check iroko.html: search for the term id in the termMap. If it is absent, add it.
53. Confirm GitHub Pages has deployed the latest push (Settings > Pages).
54. Wait 2 minutes and retry — GitHub Pages deployment is not instantaneous.

A term URI redirects to the wrong page or the wrong anchor

The iroko.html entry points to the wrong HTML file, or the anchor id in the generated HTML does not match the key in termMap.

55. Check the id attribute in the generated HTML: open the module HTML file and search for the term slug.
56. The id format is: concept-[slug] for concepts, cls-[ClassName] for classes, prop-[propertyName] for properties.
57. Compare the HTML id to the termMap key. They must match exactly.
58. Correct the iroko.html entry and push.

The Python script produces an error or skips a module

Most script errors come from TTL syntax errors. The script error message will include the TTL filename and the line causing the problem.

59. Run the TTL validation command (see Part 2) to identify the syntax error.
60. Fix the TTL.
61. Re-run the generation script.

If the module is skipped (SKIPPED message), the TTL stem is not in MODULE_CONFIG. Add it following Part 5.

A browse page renders without styling

The link to iroko-style.css in the generated HTML uses a relative path: ../assets/iroko-style.css. This resolves correctly when the HTML file is in vocab/ and the CSS is in assets/. If the directory structure has changed, the relative path will break.

The generated HTML always uses this path. To change the CSS path, change the path string in generate_vocab_html.py (search for "../assets/iroko-style.css") and regenerate all modules.

The custom domain stops resolving

In order of likelihood:

62. The CNAME file was accidentally deleted. Restore it with a single line:
ontology.irokosociety.org
63. The GitHub Pages custom domain setting was cleared. Re-enter the domain in Settings > Pages.
64. The DNS CNAME record at the domain registrar was changed or expired. Verify the record points to iroko-framework.github.io
65. GitHub Pages is having a platform incident. Check githubstatus.com

Anchor navigation does not scroll to the correct term

This is the symptom of the overflow: hidden bug described in Vol 2 Part 6 and this volume Part 6. Verify that iroko-style.css has overflow: clip (not overflow: hidden) on the .scheme-block class. If it was reverted, restore it and push.

Appendix A: Quick Reference

File locations

File	Location
TTL vocabulary files	vocab/iroko-[module].ttl
Generated HTML browse pages	vocab/iroko-[module].html
Full term index	vocab/iroko-termlist.html
Term redirect	iroko.html (repo root)
Shared stylesheet	assets/iroko-style.css
Logo	assets/IHS-Logo.jpg
HTML generator script	scripts/generate_vocab_html.py
Index generator script	scripts/generate_vocab_index.py
White paper	whitepaper/index.html
CNAME	CNAME (repo root)

Common commands

```
# Validate a TTL file
python3 -c "from rdflib import Graph; g = Graph(); g.parse('vocab/iroko-ewe.ttl',
format='turtle'); print(len(g), 'triples')"
```

```

# Regenerate one module browse page
python3 scripts/generate_vocab_html.py iroko-ewe

# Regenerate multiple module browse pages
python3 scripts/generate_vocab_html.py iroko-ewe iroko-core

# Regenerate all 16 module browse pages
python3 scripts/generate_vocab_html.py

# Regenerate the full term index
python3 scripts/generate_vocab_index.py

# Install rdflib if needed
pip3 install rdflib --break-system-packages

# Standard git commit after a vocabulary update
git add vocab/iroko-ewe.ttl vocab/iroko-ewe.html vocab/iroko-termlist.html
iroko.html
git commit -m "ewe: add iroko:medicinal-wound-healing concept"
git push origin main

```

Decision guide: what to run for each change type

Change type	Run scripts?	Update iroko.html?
Add/modify concept definition in existing TTL	generate_vocab_html.py [module] generate_vocab_index.py	Yes — add new concepts only
Fix a typo in an existing term label or definition	generate_vocab_html.py [module]	No — term id unchanged
Add a new property or class to existing TTL	generate_vocab_html.py [module] generate_vocab_index.py	Yes — for each new term
CSS change only	None	No
Add a new module	generate_vocab_html.py [module] generate_vocab_index.py (+ update MODULE_CONFIG and MODULES list first)	Yes — for every term in new module
Update whitepaper only	None	No
Fix iroko.html redirect entry only	None	Yes — that's the change

Part 10: Releases, Version Tags, and Zenodo Archiving

Overview

A deployment (Part 8) is the act of pushing changes to the live site. A release is a named, versioned snapshot of the repository that declares a stable published state of the vocabulary. These are related but distinct. A deployment happens every time any file changes. A release happens when a vocabulary version is complete and ready for formal citation, Zenodo archiving, and downstream adoption.

The Iroko Framework uses semantic versioning (MAJOR.MINOR.PATCH) consistent with the OWL versionInfo values already embedded in the TTL files. The current version is 2.0.0, issued February 2026. The versioning convention is:

- **MAJOR** — a breaking change. A namespace change, a slug rename, or the removal of a term that external datasets may already reference. The 1.x to 2.0.0 migration was a MAJOR version: 16 TTL modules were consolidated into a unified flat namespace and 1,044 terms were updated.
- **MINOR** — new terms added, new modules added, new concept schemes added. Non-breaking. External datasets referencing existing terms are unaffected.
- **PATCH** — corrections to definitions, labels, or metadata without adding or removing terms. Non-breaking.

RULE: A release is cut only when the vocabulary is in a stable, validated state. Do not cut a release mid-cycle when terms are being added or revised. The release tag represents a state that external adopters can cite and depend on.

Step 10.1 — Update version metadata in the TTL files

Before cutting a release, the version metadata in the affected TTL files must be updated to reflect the new version number. The metadata is in the ontology declaration block at the top of each TTL file:

```
# In each updated TTL file — update these three lines:
owl:versionInfo "2.1.0" ;
dcterms:issued "2025-02-01" ; # leave unchanged — this is the original issue
date
dcterms:modified "2026-04-15" ; # update to today's date
```

For a MINOR or PATCH release that touches only some modules, update versionInfo and dcterms:modified only in the modules that changed. For a MAJOR release that affects the framework as a whole, update all 16 active modules.

After updating the TTL metadata, regenerate the HTML for every updated module. The version number displayed in the browse page header ("v2.1.0") is read from OWL versionInfo by the generation script.

Step 10.2 — Run the full generation and validation pass

Before cutting a release tag, run the complete generation sequence across all updated modules and verify the output:

```
# Regenerate all updated module browse pages
python3 scripts/generate_vocab_html.py iroko-ewe iroko-core # list each updated
module

# Regenerate the full term index
python3 scripts/generate_vocab_index.py

# Verify TTL parse counts (should match expected term counts)
python3 -c "
from rdflib import Graph
import glob
for f in sorted(glob.glob('vocab/*.ttl')):
    g = Graph()
    g.parse(f, format='turtle')
    print(f'{f}: {len(g)} triples')
"
```

Spot-check the generated HTML for any updated modules. Confirm version numbers display correctly. Confirm new term URIs redirect correctly via iroko.html. Run the Vol 4 Appendix B release checklist before proceeding.

Step 10.3 — Commit the release state

All changes must be committed and pushed to the main branch before the release tag is applied. The release tag points to a commit, and that commit must contain the final state of every file that is part of the release.

```
# Stage all changed files
git add vocab/iroko-ewe.ttl vocab/iroko-ewe.html
git add vocab/iroko-core.ttl vocab/iroko-core.html # if core was updated
git add vocab/iroko-termmlist.html
git add iroko.html # if new terms were added

# Commit with a descriptive message
git commit -m "v2.1.0: add medicinal-wound-healing, candombleName property"

# Push to main
git push origin main
```

NOTE: The commit message for a release commit should begin with the version number. This makes the release commit easy to identify in the repository history without opening the release notes.

Step 10.4 — Create an annotated Git tag

A Git tag marks the specific commit that constitutes the release. Use an annotated tag (`git tag -a`), not a lightweight tag. Annotated tags contain a tagger identity, a date, and a message, and are the correct type for release versioning.

```
# Create an annotated tag
git tag -a v2.1.0 -m "Iroko Framework v2.1.0"

# Push the tag to GitHub
git push origin v2.1.0

# Verify the tag exists on GitHub
# Go to: github.com/iroko-framework/iroko-framework/tags
```

The tag name must exactly match the version number in the TTL `versionInfo` fields, prefixed with `v`. A version declared as `2.1.0` in the TTL corresponds to the tag `v2.1.0`. Inconsistency between the TTL version and the tag creates confusion for downstream users who cite the vocabulary.

Step 10.5 — Create the GitHub release

A GitHub release is a human-readable wrapper around a tag. It provides release notes, a changelog, and downloadable source archives (`.zip` and `.tar.gz`). GitHub generates the source archives automatically — they are snapshots of the full repository at the tagged commit and require no manual action.

66. Go to the repository on GitHub: github.com/iroko-framework/iroko-framework
67. Click Releases in the right sidebar, then click Draft a new release.
68. In the **Choose a tag** dropdown, select the tag you just pushed (`v2.1.0`).
69. Set the **Release title** to: Iroko Framework v2.1.0
70. Write the release notes (see release notes format below).
71. Confirm the release is set to target the main branch.
72. Click **Publish release**.

After publishing, GitHub automatically generates two source archives linked from the release page: Source code (zip) and Source code (tar.gz). These are complete snapshots of the repository at the tagged commit and can be used for Zenodo archiving.

Release notes format

Release notes should be concise and structured. The audience is vocabulary adopters who need to know whether they must update anything. The format:

```
## Summary
One sentence describing the release. E.g.: Adds wound-healing and
maternal-postpartum medicinal concepts to the Ewé module.

## What's New
```

```

- iroko:medicinal-wound-healing - new concept in MedicinalUseScheme
- iroko:medicinal-maternal-postpartum - new concept in MedicinalUseScheme
- iroko:vodouName - new annotation property on EwePlantRecord
- iroko:candombleName - new annotation property on EwePlantRecord

## Breaking Changes
None.

## Corrections
- access-initiated-elder: corrected slug (was incorrectly documented as
  access-practitioner-access in Vol 2 v1; TTL was always correct)

## Updated Modules
- iroko-ewe (v2.1.0)
- iroko-core (v2.0.1)

## Zenodo DOI
https://doi.org/10.5281/zenodo.XXXXXXX

```

Breaking changes require explicit notice and must be listed even if the list is short. For a MAJOR release, the breaking changes section should include migration guidance: what slug or namespace changed, and what adopters need to update in their own datasets.

Step 10.6 — Zenodo archiving

The Zenodo archive creates a permanent DOI-backed snapshot of the vocabulary that can be cited in scholarly publications. The current white paper DOI (10.5281/zenodo.18826673) is a Zenodo record for the white paper, not the vocabulary itself. The vocabulary should have its own Zenodo record, or the white paper record should be updated to include the vocabulary TTL files as supplementary attachments.

The two approaches are:

Option A: GitHub-Zenodo webhook (recommended for ongoing releases)

If the Zenodo GitHub integration is configured, Zenodo automatically creates a new DOI-backed archive every time a GitHub release is published. The integration is set up once:

73. Log into zenodo.org with an account linked to the iroko-framework GitHub organization.
74. Go to zenodo.org/account/settings/github.
75. Find the iroko-framework/iroko-framework repository and toggle it to On.
76. From this point forward, every GitHub release automatically triggers a Zenodo archive and generates a DOI.

Zenodo creates two DOIs per repository: a concept DOI that always resolves to the latest version, and a version-specific DOI for each individual release. For citing a specific vocabulary version in a paper, use the version-specific DOI. For citing the vocabulary in general ("the Iroko Framework vocabulary"), use the concept DOI.

Option B: Manual Zenodo deposit

If the GitHub-Zenodo webhook is not configured, deposit manually after each release:

77. Go to zenodo.org and log in.
78. Click New Upload.
79. Upload the source .tar.gz archive from the GitHub release page.
80. Fill in the metadata: title (Iroko Framework v2.1.0), creators (Iroko Historical Society), description, keywords (linked data, Afro-Atlantic, SKOS, OWL), license (CC0), related identifiers (link to the GitHub release URL).
81. Submit. Zenodo issues a DOI immediately after review (usually within minutes).
82. Copy the new DOI. Update the whitepaper HTML citation_doi meta tag and the release notes with the DOI. Commit and push.

Step 10.7 — Post-release update to the whitepaper

The Iroko Framework white paper references the framework version. After a MAJOR or MINOR release, update the white paper to reflect the new version and any new modules or significant changes. For PATCH releases, a white paper update is typically not necessary.

The white paper's DOI (10.5281/zenodo.18826673) is for the white paper document itself, not the vocabulary. A new Zenodo deposit for the white paper is warranted when the white paper content changes substantially. Minor version references in the white paper can be updated and redeployed without issuing a new Zenodo DOI for the paper.

Branching strategy

The current workflow commits directly to the main branch. This is appropriate for a small team where the ED and one or two staff members are the only people making vocabulary changes. If IHS scales to a point where multiple contributors are working on vocabulary additions simultaneously, a feature branch workflow should be adopted:

- **feature/[module]-[description]** — branch for vocabulary additions. Merged to main when the vocabulary addition is validated and the release is ready.
- **fix/[description]** — branch for corrections. Merged quickly.
- **main** — always deployable. Always reflects the current live site.

Until IHS reaches that scale, direct commits to main remain appropriate. The important discipline is: do not push broken TTL to main. Validate locally before every push.

Version history and the changelog

The GitHub releases page functions as the changelog. Each release's notes constitute the change record for that version. There is no separate CHANGELOG.md file required as long as release notes are written carefully at each release. If a CHANGELOG.md is ever added to the repository, it should be generated from the release notes, not maintained separately.

For internal reference, the IHS data directory should maintain a release log file that records:


```
# Iroko Framework Release Log

v2.0.0  2026-02  Namespace migration. Unified flat namespace.
           16 modules, 1,044 terms. Full HTML browse pages deployed.
           Zenodo DOI: 10.5281/zenodo.XXXXXXX

v2.1.0  TBD      medicinal-wound-healing, candombleName, vodouName.
           Ewé and Core modules updated.
           Zenodo DOI: pending
```

Appendix B: Release Checklist

This checklist must be completed before cutting a release tag. It covers the vocabulary state, the deployment state, and the release mechanics. Complete it in sequence — the steps are ordered by dependency.

Section 1: Vocabulary State

Check	Pass	Fail
All TTL files in this release parse without errors (rdflib validation run on each)	☐	☐
OWL versionInfo updated in all modified modules	☐	☐
dcterms:modified updated in all modified modules to today's date	☐	☐
OWL versionInfo values are consistent across all modules in this release	☐	☐
All new terms have skos:prefLabel, skos:definition, and skos:inScheme (for concepts)	☐	☐
All new properties have rdfs:label, rdfs:comment, rdfs:domain	☐	☐
iroko:minimumAccessLevel assigned to all new properties that carry access governance	☐	☐
No broken triple syntax (no missing semicolons, periods, or unclosed strings)	☐	☐
Slug format is consistent with existing framework convention (hyphen-separated lowercase)	☐	☐

Section 2: HTML Generation and Deployment

Check	Pass	Fail
generate_vocab_html.py run for all modified modules — no errors	☐	☐

Check	Pass	Fail
generate_vocab_index.py run — no errors, term count looks correct	☐	☐
Version number displays correctly in generated browse page headers	☐	☐
All new term anchors present in generated HTML (id attributes exist)	☐	☐
iroko.html updated with entries for all new terms	☐	☐
All new term URIs resolve correctly via redirect (spot-checked in browser)	☐	☐
Anchor navigation scrolls correctly to terms (overflow: clip is in place)	☐	☐
All changed files staged and committed to main	☐	☐
Push to main completed — GitHub Pages deployment green	☐	☐

Section 3: Mapping Workbook and Vol 2 Sync

Check	Pass	Fail
Mapping workbook updated for any new or changed controlled vocabulary values (Vol 3 Part 9)	☐	☐
Vol 2 GREL Quick Reference updated for any new label-to-slug mappings	☐	☐
Vol 2 predicate-object mapping table updated for any new columns or predicates	☐	☐
Vol 3 slug reference tables updated if vocabulary changed	☐	☐
Vol 1 controlled vocabulary lists updated if vocabulary changed	☐	☐
If schema changed: all four volumes updated in the coordinated sequence (Vol 3 Part 9)	☐	☐

Section 4: Release Mechanics

Check	Pass	Fail
Annotated tag created with correct version number (e.g., v2.1.0)	☐	☐
Tag pushed to GitHub (git push origin v2.1.0)	☐	☐
Tag visible at github.com/iroko-framework/iroko-framework/tags	☐	☐
GitHub release drafted with correct tag selected	☐	☐
Release title matches version tag exactly (Iroko Framework v2.1.0)	☐	☐
Release notes include: summary, what's new, breaking changes, updated modules	☐	☐

Check	Pass	Fail
Breaking changes section present (even if content is "None")	☐	☐
GitHub release published	☐	☐
Source archives (.zip and .tar.gz) visible on the release page	☐	☐

Section 5: Zenodo Archiving

Check	Pass	Fail
Zenodo archive created (webhook auto-triggered, or manual deposit completed)	☐	☐
Version-specific Zenodo DOI recorded in the IHS release log	☐	☐
Release notes on GitHub updated with the Zenodo DOI	☐	☐
White paper HTML updated with new DOI if applicable	☐	☐
IHS release log file updated with version, date, and DOI	☐	☐

Sign-Off

Release Version	Date	Executive Director
Zenodo DOI:		
Notes:		

NOTE: File the completed checklist as: *Release_Checklist_[Version]_[Date].pdf* in the IHS records alongside the dataset QC checklists from Volumes 1 and 2.